

Experimental Implementation of Remote Attestation over OPC UA Protocol

Arto Juhola

VTT Technical Research Centre of Finland
Espoo, Finland
arto.juhola@vtt.fi

Markku Kylänpää

VTT Technical Research Centre of Finland
Espoo, Finland
markku.kylanpaa@vtt.fi

Abstract—We present design alternatives and a proof-of-concept demonstration realising the inclusion of remote attestation into Open Platform Communication — Unified Architecture (OPC UA), a modern Operational Technology (OT) protocol. This aspect is not yet covered by the OPC UA set of specifications. Remote attestation is about ascertaining the integrity of the software and the configuration of the attested devices, and is considered to be essential for safeguarding critical infrastructure. Remote attestation and OPC UA concepts are briefly summarized. We discuss alternatives for realising remote attestation over OPC UA, and, based on their relative merits, select one for experimentation. Related work is also discussed. Python-based proof-of-concept implementation was realised and experimented with for unearthing relevant issues that need to be resolved for actual OPC UA remote attestation. The findings are presented and the demonstrator is available as open source software as an example how to implement remote attestation over OPC UA protocol.

Index Terms—remote attestation, OPC UA, proof-of-concept demonstration

I. INTRODUCTION

Industrial systems are gaining unwelcome interest from the part of cyber criminals and hostile states. The need to protect these systems is recognised and modern industrial automation communication protocols are offering means for improved security. An area not yet covered by up-and-coming protocols is the support for remote attestation, but there are proposals to introduce this functionality [1]. We describe a proof-of-concept experimentation aiming for the practical realisation of remote attestation into OPC UA. The choice of OPC UA was motivated by its potential of becoming the “lingua franca” of “Industry 4.0” [2]. The protocol was also a target of recent Pipedream attack [3], [4]. OPC UA provides communications security, cross-platform support and versatility to cover both horizontal and vertical OT communication needs. OPC UA is an object-oriented protocol, supporting client-server and publish-subscribe modes of operation. For the remote attestation, the client-server mode is used, with additional interfaces and functionality.

The rest of the paper is structured as follows. First, in the section II a brief introduction to remote attestation concepts is given. The section III contains a short introduction to OPC UA. The design choices and related work are described in the section IV and the current implementation is described in the

section V. Remaining future work is discussed in the section VI and conclusions are in the section VII.

II. REMOTE ATTESTATION CONCEPTS

Remote attestation is an activity of making a claim about properties of a target by supplying evidence to an appraiser over a network [5]. Making trustworthy claims requires that there is an isolated measurement mechanism for system integrity and a root of trust that allows protection of these measurements. Measurement mechanism is typically collecting integrity measurements by calculating a cryptographic hash of the loaded components in a process called trusted boot, which also protects collected measurements against manipulation by using a hardware-based security module as described in [6]. A remote client can request attestation from the system by sending an *attestation request* message to the system. The system replies with an *attestation reply* message that includes an *attestation evidence* containing the measurement information and is signed by a key based on the root of trust. The key is also known as an *Attestation Identity Key* (AIK). Trustworthy measurement collection and signing requires an isolated execution environment based on a hardware-based security module. The lack of these features has prevented the use of remote attestation becoming a mainstream especially in systems including also low-end devices. However, the situation is now changing as there are hardware-based security module technologies available also to low-end devices.

Trusted Platform Module (TPM) [7] specification from Trusted Computing Group (TCG) is a classic example of a hardware-based security module that can be used in remote attestation scenarios. TPM includes Platform Configuration Registers (PCRs) that are used to record boot sequence measurements. These registers do not support direct assignment. The only update mechanism is called TPM extend that is concatenating the input value with the old value of the register and then calculates a cryptographic hash of the concatenated value and finally assigns the result to the register. TPM also stores a key that can be used to sign the attestation evidence that includes these PCR values. TCG has also other secure hardware technologies that can be used with remote attestation and are more suitable for micro-controller class devices. Device Identifier Composition Engine (DICE) [8] implements layered architecture deriving keys and device certificates from

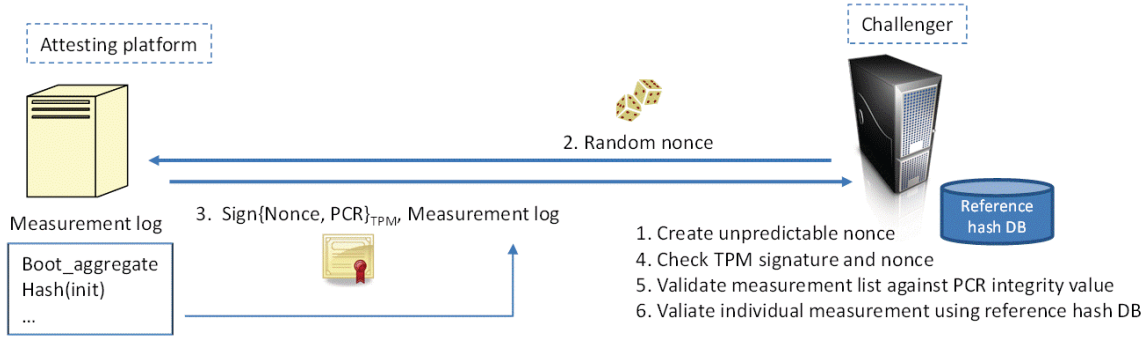


Fig. 1. TPM-based remote attestation scenario.

device secret. Low-end ARM devices that support TrustZone technology [9] adopted for low-end devices can use ARM Platform Security Architecture (PSA) Attestation [10].

Typical remote attestation scenario is described in figure 1. The challenger is sending an attestation request to a system to be attested. The request includes an unpredictable nonce that is used to guarantee freshness of the attestation reply message and to prevent record and playback attacks. The system to be attested is expected to collect integrity measurements during system startup. These measurements must be integrity protected using an isolated execution environment that also contains a signing key (AIK). Isolation of the isolated execution environment must be based on hardware-based security. TPM and ARM TrustZone are examples of hardware-based isolated execution environments that can be used to protect integrity measurements. Integrity measurements must be collected using *measured boot* meaning that during boot and system startup each executable component must be measured before passing control to it. The attestation reply message contains *attestation evidence*. When using TPM it is enough to sign a data structure containing the received nonce value and TPM PCR values. The signed blob is passed to the attestation reply message together with an unsigned measurement log.

When the challenger receives the attestation reply, the message must be verified. At first, the signature should be checked using a certified public key. If TPM is used, the measurement log measurements should be applied to recalculate the PCR values that should match to those that are received in the signed blob of the attestation reply message. If there is a match then the measurement log is considered to be correct. However, individual measurements should still be checked against expected values that are stored in the reference hash database.

III. OPC UA

OPC UA is a large set of specifications specified by the OPC Foundation. Specifications are also standardized as IEC standard (IEC 62541). It is also selected as a communication backbone for German Industrie 4.0 initiative [2]. The goal

has been to specify platform independent protocols and information models that can be used for reliable data exchange in industrial automation. OPC UA includes object-oriented information model and supports multiple messaging types, e.g., both publish-subscribe and client-server communication types [11]. It is also possible to configure alarms and other asynchronous events. Security and extensibility are also goals of the specifications.

OPC UA server makes its services, data types, and content available to clients as a model called *AddressSpace*. OPC UA also allows data to be exposed in many different formats and different hierarchies. It is also possible to use established messaging protocols as a transport layer and have content available in OPC UA compatible way. There are also many open source implementations of OPC UA in different languages and with different licenses [12].

OPC UA Connection Protocol (UACP) and OPC UA Secure Conversation (UASC) can be used to setup a secure channel providing confidentiality, integrity, and authentication of the end-points in a similar way as Transport Layer Security (TLS). OPC UA clients and servers can be certified against certain profiles specifying security functions. OPC UA also defines three different communications security modes:

- *None* - No protection against eavesdropping or message modification. This can be used during a system development phase but should not be used in production environment.
- *Sign* - Provides integrity protection for OPC UA messages by signing messages with a certified asymmetric key. The message receiver can verify integrity of the message. However, actors that are capable of monitoring network traffic can still read (but not alter) these messages.
- *SignAndEncrypt* - The messages are signed and also encrypted to provide confidentiality against eavesdroppers. The encryption key is exchanged during the secure channel setup handshake as described in figure 2.

OPC UA servers and clients define *Profiles* [14] that define supported functionality. Profiles can also define *SecurityPolicy*

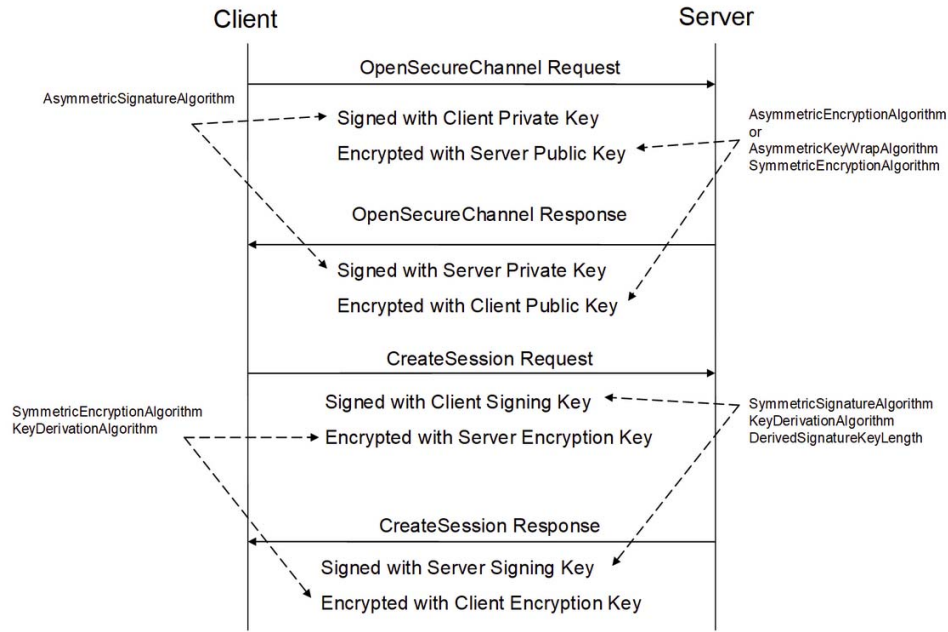


Fig. 2. OPC UA security handshake [13].

that can specify supported cryptographic algorithms. There is also a Global Discovery Server (GDS) that can provide centralized certificate management service [15].

OPC UA security model [16] is focused on securing communications between end points. The threat model includes the concept of rogue servers but only certificate check is mentioned as reconciliation of these threats. Problems with bootstrapping initial trust are discussed in [17]. There is no discussion about the use of remote attestation for integrity verification of the remote end point in OPC UA specifications. If OPC UA is used in critical infrastructure, it is necessary to define mechanisms how remote attestation can be used with OPC UA.

IV. DESIGN ALTERNATIVES AND RELATED WORK

The presented experiment is concerned about the client-server communications only. The OPC UA standard has provisions for the arrangement of the communications security (sign and encrypt), and the underlying supposition in the following considerations is that the strongest practicable measures of the standard are chosen in the connection setup. As for the remote attestation, there are at least the following design alternatives:

- The incorporation of the attestation messaging in the connection setup process, manifested by a special security mode with remote attestation, available for selection by the client.
- Uni-directional attestation, where the server can fend off replay-attacks by relying on secure time-stamps instead of "nonces" used with bidirectional attestation [18]. Since the communications flow from a server to a client only,

attestations will be sent periodically (one minute interval is suggested for most cases in [18]).

- Incorporation of a special attestation object in the server interface. The client can request attestation by sending a method call to the object [1].

Combining remote attestation with connection setup would save some messaging overhead, since the negotiation of the security features would include an option for remote attestation. This was however not thought to be a significant benefit, especially when means for launching attestation at other times would still be needed.

TCG has a set of specifications called Trusted Network Connect (TNC) [19] that can be used to integrate remote attestation to setting up a TLS-like secure connection. Another example is from Knauth et al. [20] where Intel SGX-based remote attestation has been integrated with a TLS connection setup. IETF RATS working group is also working on to develop Remote Attestation Procedures Architecture [21]. They note that existing protocols already have mechanisms for passing security information for authentication and authorization purposes and propose the use of X.509 certificate extension to carry remote attestation information. OPC UA protocol was mentioned as an example.

Additionally, OPC UA session can be run over HTTPS connection [13] and underlying TLS-connection could be used to carry remote attestation information during session connection. However, it may be difficult to utilize integrity verification from upper layers causing difficulties in error handling.

The unidirectional case was deemed to be a special case,

since usually the client-server communications are bidirectional. Also, the required reliable and secure timekeeping is not necessarily available in the devices to be attested.

The server interface with an attestation object offers flexibility, since there are cases where attestation might be wished to either omit (say, repeated connection attempts when recovering from a connection loss) or execute at times other than connection setup (say, collecting evidence for a successful accomplishment of some activity, e.g., for contractual reasons).

Certificate management and reference integrity metrics is an essential part of the remote attestation systems. Birnstill et al. [1] refer to OPC UA GDS certificate management as a way to manage certificates. There should also be a software inventory containing reference integrity information for all installed software. The inventory must be updated and must be in sync when new software versions are installed in the devices.

V. EXPERIMENTAL IMPLEMENTATION

Given the pros and cons of the design alternatives, the approach chosen was augmenting the server interface with an attestation object. Also, although devices with TPM were the starting point, the presented experiment – being about messaging protocol – is agnostic about the details of attestation within a device.

The sequence diagram in figure 3 presents the overall messaging related to the server attestation object, omitting attestation management aspects. The experimental implementation does not at the moment cover the initial query for available server objects, nor the possible collection of the evidence from the subsystems (and combining into the evidence covering the complete system) shown in the diagram. Note that the latter aspect is depicted in a simplified form: each subsystem needs to be queried and managed separately, some subsystems might be only indirectly reachable (via intermediates), and the composition of the system might be dynamic, like in the case of "swarms" [22].

The implementation uses FreeOpcUa opcuasyncio open-source Python libraries [23]. The opcuasyncio implements the connection security functionality of the OPC UA standard, and in the experiment the strictest security mode is used. There are other implementations of OPC UA with different programming languages and their combinations, but for the kind of experimentation with no stringent performance requirements pure Python was deemed to be the best alternative. To cater for such demanding cases where performance is an issue, switching to hybrid high level - system level language or system level languages like C++ or Rust can be made later. Also, developing with system level languages is likely to be faster with major design decisions verified initially with languages like Python. However, opcuasyncio uses modern Python3 constructs, especially coroutines, for improving performance. At the server side, the current implementation includes a server object offering a method that, when receiving an attestation request from the client, launches a coroutine that returns a dummy response after waiting a couple of seconds. As for the

future development, the coroutine can be expanded to launch actual attestation related activities.

The client implementation offers an IPython interface affording the ability to execute client functionality, with manual input and by loading files containing appropriate commands. A file was prepared containing the following sequence of commands: to get a reference to the server attestation object, to form an attestation request with a nonce, to send it to the server attestation object, wait for the response and finally to display it. The source code is available in [24].

VI. FUTURE WORK

Below is the list of the remaining work. This encompasses open questions and items requiring the attention of standardisation bodies.

- The initiation process could involve determining the availability of the remote attestation, to avoid confusing devices not having this capability. The existing protocol provides a way for a client to simply ask for the available possibilities.
- The interfaces of the server object(s) provided for remote attestation should be defined, along with their namespaces. We suggest that the names of the respective "objectTypes" (about the same as object classes in object oriented programming) should include parts to point out the associated standard organisation, the standard, its version and the object interface to be complied to.
- After obtaining the "objectType" and the instance IDs of an object, the client should need no further information for calling the object's methods.
- The size of the nonce in the attestation request message needs to be specified. The nonce is used for fending off replay-type of attacks. In asyncio-opcuasyncio, the recommended size for the nonce used in client-server connection establishment phase is recommended to be of the same size as the key used for the encryption of the said connection. This was also our initial choice.
- The transfer format of the attestation result needs to be sorted out. Since the intention was just to try out the protocol we used a "dummy" result in the response.
- The complete attestation handling involves additional steps with related messaging, e.g., the TPM case is discussed in [25].
- Handling of exceptional situations requires defining and realisations (e.g., error messages specific to the opcuasyncio extension).
- The presence of subsystems that need to be attested as a part of the complete remote attestation needs consideration. Say, the subsystem composition might be dynamic, or they might have attestation interfaces different from each other or the one offered by the principal attestation interface.
- Remote attestation scenarios are based on client/server model but OPC UA is also used in publish/subscribe messaging. There is a need to study how remote attestation could be used also with publish/subscribe messaging.

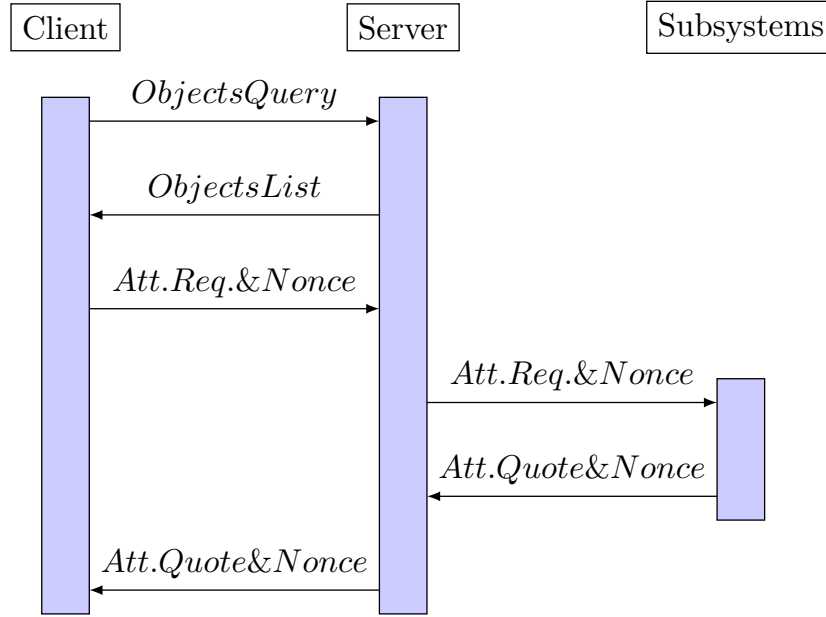


Fig. 3. Proposed attestation sequence.

- Demonstrator implementation integrated with real security hardware in an embedded platform using a system level language, e.g., C++ or Rust.
- Performance, memory requirements and messaging delay should also be analyzed.
- Large systems, having lots of nodes, need to be split into subsystems that can be connected through a gateway node. Property-based attestation can be considered as a way to make integrity claims about the subsystem [26].
- Ongoing standardization efforts in IETF RATS WG is expected to standardize formats for describing assertions/claims about system components and associated evidence that could also be used when implementing more sophisticated remote attestation mechanisms for OPC UA.

VII. CONCLUSIONS

Remote attestation has not so far been taken into account in OPC UA specifications although it should be an essential part of the critical infrastructure systems. However, OPC UA provides mechanisms that can be used to implement remote attestation support. A couple of alternatives are considered but the most promising one is to augment the server interface to contain functionality to support remote attestation.

The presented implementation is a proof-of-concept reality check but shows that OPC UA could be used for conveying remote attestation. The focus of our work is just in protocol integration, without real hardware-based security module involvement.

OPC UA standardization work should specify a framework for remote attestation as real world implementations can be large multi vendor installations and fragmentation to vendor-specific solutions would limit utilization of the feature.

ACKNOWLEDGMENT

The work presented here has been carried out in the Secure Collaborative Intelligent Industrial Assets (SeCoIIA) project from the European Union's Horizon 2020 programme under grant agreement No 871967 and the ITEA project 17032 CyberFactory#1.

REFERENCES

- [1] P. Birnstill, C. Haas, D. Hassler, and J. Beyerer, "Introducing remote attestation and hardware-based cryptography to OPC UA," in *22nd IEEE International Conference on Emerging Technologies and Factory Automation, ETFA 2017, Limassol, Cyprus, September 12-15, 2017*. IEEE, 2017, pp. 1–8. [Online]. Available: <https://doi.org/10.1109/ETFA.2017.8247591>
- [2] S. Hoppe, "There is no Industrie 4.0 without OPC UA," White paper <https://opconnect.opcfoundation.org/2017/06/there-is-no-industrie-4-0-without-opc-ua/>, OPC Foundation., Tech. Rep., 2017, accessed: 2022-04-18.
- [3] "APT cyber tools targeting ICS/SCADA devices," Alert (AA22-103A) <https://www.cisa.gov/uscert/ncas/alerts/aa22-103a>, Cybersecurity and Infrastructure Security Agency (CISA), Tech. Rep., 2022, accessed: 2022-04-17.
- [4] "Pipedream: Chernovite's emerging malware targeting industrial control systems," White paper https://hub.dragos.com/hubfs/116-Whitepapers/Dragos_ChernoviteWP_v2b.pdf, Dragos Inc., Tech. Rep., 2022, accessed: 2022-04-17.
- [5] G. Coker, J. D. Guttman, P. A. Loscocco, A. L. Herzog, J. K. Millen, B. O'Hanlon, J. D. Ramsdell, A. Segall, J. Sheehy, and B. T. Sniffen, "Principles of remote attestation," *Int. J. Inf. Sec.*, vol. 10, no. 2, pp. 63–81, 2011. [Online]. Available: <https://doi.org/10.1007/s10207-011-0124-7>

- [6] R. Sailer, X. Zhang, T. Jaeger, and L. van Doorn, "Design and implementation of a TCG-based integrity measurement architecture," in *13th USENIX Security Symposium (USENIX Security 04)*. San Diego, CA: USENIX Association, Aug. 2004. [Online]. Available: <https://www.usenix.org/conference/13th-usenix-security-symposium/design-and-implementation-tcg-based-integrity-measurement>
- [7] W. Arthur and D. Challener, *A Practical Guide to TPM 2.0: Using the Trusted Platform Module in the New Age of Security*, 1st ed. USA: Apress, 2015.
- [8] Trusted Computing Group, "Hardware requirements for a device identifier composition engine, level 00 revision 78," 2018.
- [9] S. Pinto and N. Santos, "Demystifying Arm TrustZone: A comprehensive survey," *ACM Comput. Surv.*, vol. 51, no. 6, jan 2019. [Online]. Available: <https://doi.org/10.1145/3291047>
- [10] ARM, "PSA Attestation API 1.0," 2020.
- [11] "OPC Unified Architecture Specification Part 1: Overview and Concepts," OPC Foundation, Tech. Rep. 1.04ed., November 2017.
- [12] "List of Open Source OPC UA Implementations," accessed: 2022-04-18. [Online]. Available: <https://github.com/open62541/open62541/wiki/List-of-Open-Source-OPC-UA-Implementations>
- [13] "OPC Unified Architecture, Part 6: Mappings," OPC Foundation, Tech. Rep. 1.04ed., November 2017.
- [14] "OPC Unified Architecture, Part 7: Profiles," OPC Foundation, Tech. Rep. 1.04ed., November 2017.
- [15] "OPC Unified Architecture, Part 12: Discovery and Global Services," OPC Foundation, Tech. Rep. 1.04ed., February 2018.
- [16] "OPC Unified Architecture, Part 2: Security Model," OPC Foundation, Tech. Rep. 1.04ed., March 2018.
- [17] D. Meier, F. Patzer, M. Drexler, and J. Beyerer, "Portable trust anchor for OPC UA using auto-configuration," in *2020 IEEE 25th International Conference on Emerging Technologies and Factory Automation (ETFA) : Technical University of Vienna, Vienna, Austria, 08-011September 2020 : proceedings / organized by: IEEE Industrial Electronics Society (IES)*, vol. 1, 2020, p. 270–277.
- [18] H. Birkholz, A. Fuchs, I. McDonald, and C. Bormann, "Time-based uni-directional attestation," <https://datatracker.ietf.org/doc/html/draft-birkholz-tuda-04>, 2017, work in progress, accessed: 2022-03-07.
- [19] "TCG Trusted Network Communications TNC architecture for interoperability," Specification Version 2.0 Revision 13 <https://trustedcomputinggroup.org/wp-content/uploads/TCG-TNC-Architecture-for-Interoperability-Version-2.0-Revision-13-.pdf>, Trusted Computing Group, Tech. Rep., 2017, accessed: 2022-04-20.
- [20] T. Knauth, M. Steiner, S. Chakrabarti, L. Lei, C. Xing, and M. Vij, "Integrating remote attestation with transport layer security," *CoRR*, vol. abs/1801.05863, 2018. [Online]. Available: <http://arxiv.org/abs/1801.05863>
- [21] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan, "Remote attestation procedures architecture," <https://datatracker.ietf.org/doc/draft-ietf-rats-architecture/>, 2022, work in progress, accessed: 2022-04-20.
- [22] N. Asokan, F. Brasser, A. Ibrahim, A.-R. Sadeghi, M. Schunter, G. Tsudik, and C. Wachsmann, "SEDA: Scalable embedded device attestation," in *CCS '15 Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. United States: ACM, Association for Computing Machinery, 2015, pp. 964–975. [Online]. Available: <http://www.sigsac.org/ccs/CCS2015/>
- [23] FreeOpcUa, "FreeOpcUa opcua-asyncio," 2020, accessed: 2022-03-08. [Online]. Available: <https://github.com/FreeOpcUa/opcua-asyncio>
- [24] A. Juhola and M. Kylänpää, "pythonOpcua," 2022, accessed: 2022-03-09. [Online]. Available: <https://github.com/ttemjk/pythonopcua>
- [25] I. Desai, "Remote attestation with tpm2 tools," <https://tpm2-software.github.io/2020/06/12/Remote-Attestation-With-tpm2-tools.html>, 2020, accessed: 2022-03-09.
- [26] A.-R. Sadeghi and C. Stübke, "Property-based attestation for computing platforms: Caring about properties, not mechanisms," in *Proceedings of the 2004 Workshop on New Security Paradigms*, ser. NSPW '04. New York, NY, USA: Association for Computing Machinery, 2004, p. 67–77. [Online]. Available: <https://doi.org/10.1145/1065907.1066038>